For Loops

Syntactic Sugar

We could add a new `For` AST node and update our interpreter to evaluate it, just like we did for `while`

But `for` loops are fundamentally just a `while` loop with some extra structure wrapped around it

We can use desugaring

> translating this "syntactic sugar" into simpler constructs our parser already knows how to handle

Because we are desugaring, we don't need to add a new node to `tool/GenerateAst.java` !

Updating the Grammar

We add the `for` statement to our grammar

A `for` loop has three optional clauses inside the parentheses: an initializer, a condition, and an increment

```
statement  → exprStmt
           | forStmt
           | ifStmt
           | printStmt
           | whileStmt
           | block ;

forStmt    → "for" "(" ( varDecl | exprStmt | ";" )
             expression? ";"
             expression? ")" statement ;
```

Updating the Parser

Over in the parser, we add the match for the keyword in our `statement()` rule

```
lox/Parser.java
in statement()
```

```
    if (match(FOR)) return forStatement();
    if (match(IF)) return ifStatement();
    if (match(PRINT)) return printStatement();
```

Parsing the Clauses

The `forStatement()` method needs to parse all three optional clauses and the body

```
lox/Parser.java
after whileStatement()
```

```
private Stmt forStatement() {
    consume(LEFT_PAREN, "Expect '(' after 'for'.");

    Stmt initializer;
    if (match(SEMICOLON)) {
        initializer = null;
    } else if (match(VAR)) {
        initializer = varDeclaration();
    } else {
        initializer = expressionStatement();
    }
```

Parsing the Clauses

```
lox/Parser.java
after whileStatement()
```

```
    ...
    Expr condition = null;
    if (!check(SEMICOLON)) {
        condition = expression();
    }
    consume(SEMICOLON, "Expect ';' after loop condition.");

    Expr increment = null;
    if (!check(RIGHT_PAREN)) {
        increment = expression();
    }
    consume(RIGHT_PAREN, "Expect ')' after for clauses.");

    Stmt body = statement();
```

Desugaring into While

Now that we have all the parsed pieces, we *assemble* them into standard AST nodes

We construct the desugared AST from the *bottom up*, starting with the body and the increment, then wrapping it in a `while` loop, and finally appending the initializer

Desugaring the Increment

```
lox/Parser.java
in forStatement(), continued
```

```
    if (increment ≠ null) {
        body = new Stmt.Block(
            Arrays.asList(
                body,
                new Stmt.Expression(increment)));
    }

    if (condition == null) {
        condition = new Expr.Literal(true);
    }
    body = new Stmt.While(condition, body);

    if (initializer ≠ null) {
        body = new Stmt.Block(Arrays.asList(initializer, body));
    }

    return body;
}
```

Execution

That’s it

Because we transformed the `for` loop into standard `Block` and `While` nodes during the parsing phase, we don’t need to write any new code in `Interpreter.java`

Our interpreter already knows exactly how to execute blocks and while loops, so it handles our desugared for loops flawlessly

Exercise

After implementing the code

Make a program that calculates the first 10 digits of the fibonnaci sequence using a `for` and a `while` loop

Remember that the formula for the fibonnaci sequence is:

$$F(n) = F(n-1) + F(n-2)$$

name the file `fibonacci_lastname.lox`